

Database 2

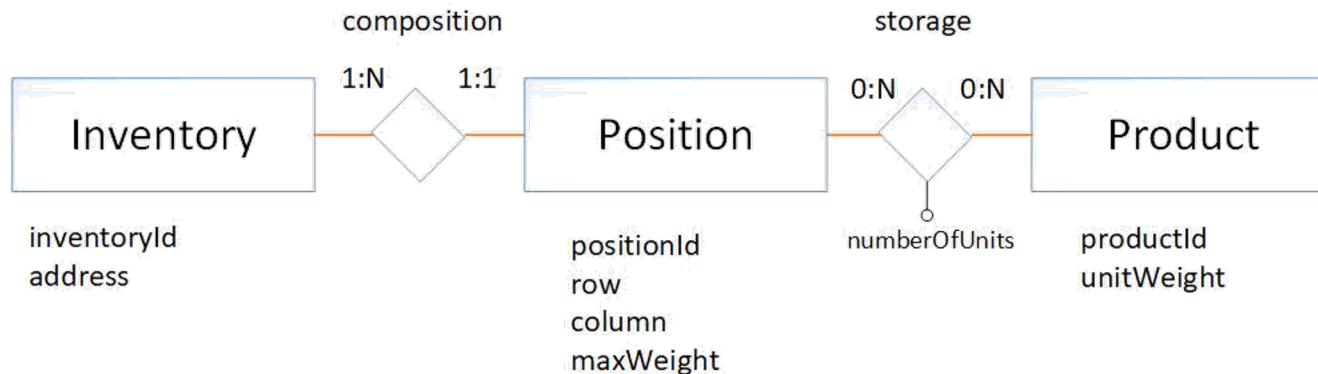
JPA exercise

Exam on Feb 4 2022

Solutions

Specifications

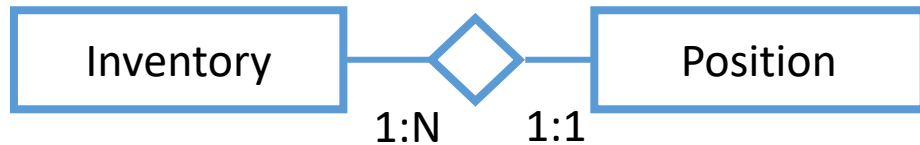
An application manages the storage of products in a repository. The data are organized according to the following conceptual model:



The user can inspect any one of the inventories and browse the positions. By selecting a position the user can check the current status of occupancy (empty or occupied by one or more products, total weight of the occupants, etc). The user can also search for a productId and see the positions where it is stored and the inventory corresponding to each such position. Products are in the order of hundreds of thousand, inventories are in the order of ten and positions per inventory in the order of thousands.

- Show the JPA entities (with all their attributes) that map the domain objects of the conceptual model, taking into account the above mentioned access paths of the application and data cardinalities.
When designing the annotations for the relationships, specify the owner side of the relationship, the mapped-by attribute, the fetch policy and the cascading policies you consider more appropriate to support the access required by the web application.
- Comment the design choices you have made.

Relationship composition



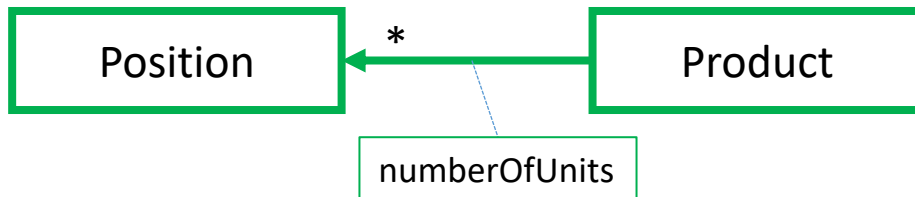
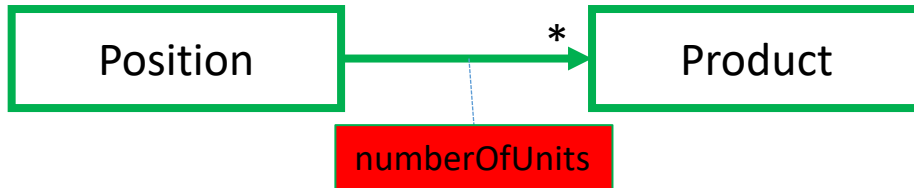
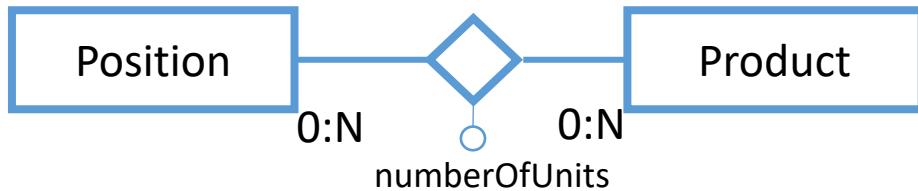
- Inventory → Position
@OneToMany

- Necessary to browse the positions of an inventory
- Owner = Position
- FetchType is LAZY (positions per inventory are in the order of thousands)
- Remove can be cascaded

- Position → Inventory
@ManyToOne

- necessary to show the inventory where a product is available after a search through its position (in alternative using a named query is possible)
- FetchType can be EAGER (one inventory per position)
- No cascade is needed

Relationship storage



- Position → Product
@ManyToMany
 - needed to show all the products stored in a position
 - Owner = product or position
 - FetchType can be EAGER (products per positions are in order of ten)
 - It must support the retrieval of the numberOfUnits of each product in a position
 - No cascade is needed
- Product → Position
@ManyToMany
 - necessary to show the inventory where a product is available after a search through its position (in alternative using a named query is possible)
 - FetchType can be EAGER (positions per product are few)
 - No cascade is needed

Entity Inventory

```
@Entity
public class Inventory implements Serializable {
    @Id @GeneratedValue(strategy = GenerationType.AUTO)
    private int inventoryId;
    private String address;

    //LAZY can be omitted, default FetchType for collections
    // REMOVE can be cascaded
    @OneToMany(mappedBy="inventory", fetch = FetchType.LAZY,
               cascade = CascadeType.REMOVE)
    private List<Position> positions;

    // getters setters and constructors
    // . . .
```

Entity Position

```
@Entity
public class Position implements Serializable {
    @Id @GeneratedValue(strategy = GenerationType.AUTO)
    private int positionId;
    private int row;
    private int column;
    private int maxWeight;

    @ManyToOne(fetch = FetchType.EAGER) //Eager can be omitted, default value
    @JoinColumn(name="inventory") //owner of the relation
    private Inventory inventory;

    @ElementCollection(fetch = FetchType.EAGER)
    @CollectionTable(name = "storage",
                     joinColumns = @JoinColumn(name = "positionId"))
    @MapKeyJoinColumn(name = "productId") --this is the map key, no inverseJoinColumn
    @Column(name = "numberOfUnits")
    private Map<Product, Integer> products;

    // getters setters and constructors
    // . . .
```

Entity Product

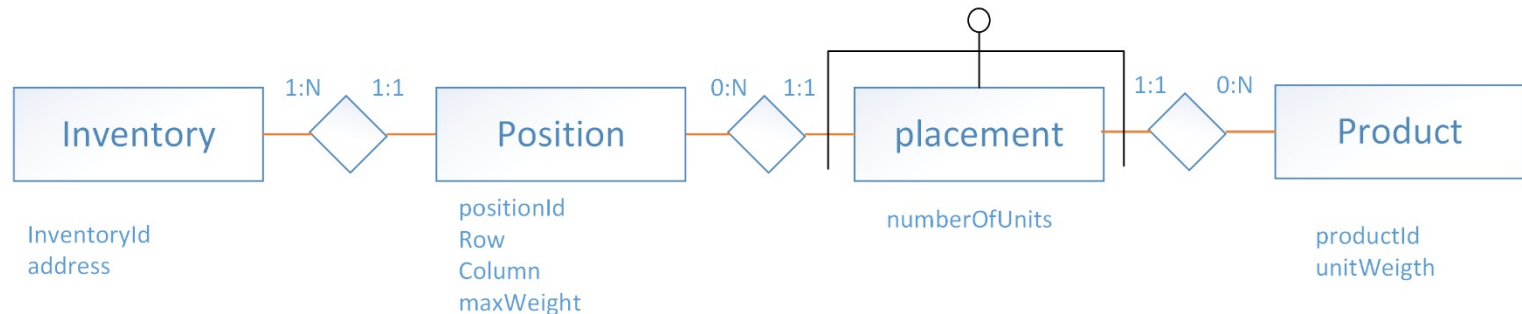
```
@Entity
public class Product implements Serializable {
@Id @GeneratedValue(strategy = GenerationType.AUTO)
    private int productId;
    private int unitWeight;

    @ManyToMany (fetch = FetchType.EAGER)
        @JoinTable(name = "storage",
            joinColumns = @JoinColumn(name = "productId"),
                inverseJoinColumns = @JoinColumn(name ="positionId"))
    private List<Position> positions; // owner of the relation

// getters setters and constructors
// . . .
```

Alternative solution

- Mapping N:M relationship with attribute without using @ElementCollection
- You can map using regular relationship annotations by objectifying the N:M relationship with attribute
- The following schema can be mapped



- The *placement* weak entity is given a composite key due to the external identification by the two 1:1 relationships

Entity Position / alternative

```
@Entity
public class Position implements Serializable {
    @Id @GeneratedValue(strategy = GenerationType.AUTO)
    private int positionId;
    private int column;
    private int row;
    private int maxWeight;

    @ManyToOne(fetch = FetchType.EAGER) //Eager can be omitted, default
    @JoinColumn(name="inventory") //owner of the relation
    private Inventory inventory;

    @OneToMany(mappedBy = "position", fetch=FetchType.EAGER)
    private List<Placement> placements;

    // getters setters and constructors
    // . . .
```

Entity PlacementId / alternative

```
@Embeddable // this implements values of a composite key
public class PlacementId implements Serializable {

    private Long positionId; // members of the composite key
    private Long productId;

    public PlacementId() {

    }

    public PlacementId(Long positionId, Long productId) {
        super();
        this.positionId = positionId;
        this.productId = productId;
    }

    // setters, getters, hashCode, equals
}
```

Entity Placement / alternative

```
@Entity
public class Placement implements Serializable{

    @EmbeddedId
    private PlacementId id;

    @ManyToOne(fetch=FetchType.EAGER) //Eager can be omitted
    @MapsId("positionId") // reference to the foreign key attribute
    @JoinColumn(name = "positionId")
    private Position position;

    @ManyToOne(fetch=FetchType.EAGER) //Eager can be omitted
    @MapsId("productId") // reference to the foreign key attribute
    @JoinColumn(name = "productId")
    private Product product;

    private int numberOfUnits;

    // getters, setters, ...
}
```

Entity Product / alternative

```
@Entity
public class Product implements Serializable {
    @Id @GeneratedValue(strategy = GenerationType.AUTO)
    private int productId;
    private int unitPrice;

    @OneToMany(mappedBy = "product", fetch=FetchType.EAGER)
    private List<Placement> placements;

    // getters setters and constructors
    // . . .
```